

ITNE352 Project – Group A4

Recipe Finder: Multi-Threaded Client-Server Application

Project Description

This project is a multi-threaded client-server application built in Python that allows multiple users to search and browse recipes in real time. The server connects to [TheMealDB](#) API to fetch recipe data and handles multiple simultaneous client connections using threads. Each client gets an interactive terminal menu to search, filter, and browse recipes by name, category, area, or ingredient. Every recipe request made by a client is automatically saved as a JSON file on the server side for testing and evaluation purposes.

The project demonstrates key networking concepts including TCP socket programming, length-prefixed message framing, multi-threading, and a hybrid data-handling pattern that combines a startup reference cache with on-demand API fetching.

Semester

Semester 2 – 2025/2026

Group

Field		Details
Group Name		Group A4
Course Code		ITNE352
Section		Section 1

#	Student Name	Student ID
1	Layla Khalil	202302358
2	Mohammed Jamal	202302745

Table of Contents

1. [Project Description](#)
2. [Semester](#)
3. [Group](#)
4. [Requirements](#)
5. [How To Run](#)
6. [The Scripts](#)
7. [Additional Concept](#)
8. [Acknowledgments](#)
9. [Conclusion](#)
10. [Resources](#)

Requirements

Python Version

Python 3.8 or higher is required. Check your version by running:

```
python --version
```

Download Python from: <https://www.python.org/downloads/>

Dependencies

This project uses **only Python standard library modules** — no external packages need to be installed:

Module	Purpose
<code>socket</code>	TCP client-server communication
<code>json</code>	Encoding and decoding messages
<code>urllib.request</code>	HTTP requests to TheMealDB API
<code>threading</code>	Handling multiple clients simultaneously
<code>os</code>	File path handling for saving JSON files

Internet Connection

The server requires an active internet connection to fetch data from TheMealDB API at:

```
https://www.themealdb.com/api/json/v1/1
```

Setup Steps

1. Make sure Python 3.8+ is installed
2. Clone the repository:

```
git clone https://github.com/MohdJamal05/ITNE352-Project-Group-A4.git
```

3. Navigate into the project folder:

```
cd ITNE352-Project-Group-A4
```

4. No additional installation needed — the project is ready to run

How To Run

Step 1 — Start the Server

Open a terminal in the project folder and run:

```
python server.py
```

The server will:

- Fetch categories, areas, and ingredients from TheMealDB
- Save them to `reference_A4.json`
- Start listening for clients on `127.0.0.1:12000`

Expected output:

```
Fetching reference data from TheMealDB...
Reference data saved to reference_A4.json
Server listening on 127.0.0.1:12000...
```

Step 2 — Start the Client

Open a **second terminal** in the same folder and run:

```
python client.py
```

Enter your name when prompted. The server will greet you and the main menu will appear.

Step 3 – Interact with the Menus

Main Menu

1. Browse recipes
2. Reference lists
3. Quit

Recipes Menu options:

- Search by name – type any keyword
- Filter by category – choose from: Beef, Chicken, Seafood, Vegetarian, Dessert, Pasta, Breakfast
- Filter by area – choose from: Italian, Indian, Mexican, Japanese, Moroccan, British, American, Thai
- Filter by ingredient – type any single ingredient
- Random recipe – returns a full recipe instantly with no list step

Reference Menu options:

- List all categories
- List all areas
- List all ingredients (first 50)

Multiple Clients

Open as many terminals as needed and run `python client.py` in each – the server handles them all simultaneously using threads.

Output JSON Files

After each recipe request, a JSON file is saved in the project folder:

```
<username>_<option>_A4.json
```

Examples:

```
mohammed_search_by_name_A4.json
mohammed_filter_by_category_A4.json
layla_random_recipe_A4.json
layla_get_detail_A4.json
```

The Scripts

`server.py`

The server script manages all client connections, fetches data from TheMealDB API, and saves results to JSON files.

Utilized packages: `socket`, `json`, `urllib.request`, `threading`, `os`

Classes

MessageHelper Handles the length-prefixed JSON communication protocol.

```
@staticmethod
def send(conn, payload: dict) -> None:
    data = json.dumps(payload).encode('utf-8')
    conn.sendall(len(data).to_bytes(4, 'big') + data)
```

Every message is prefixed with a 4-byte integer indicating its length. `_recv_exact` loops until all expected bytes are received, handling TCP fragmentation correctly.

MealDBClient Encapsulates all HTTP calls to TheMealDB API.

```
def fetch(self, url: str) -> object:
    try:
        with urllib.request.urlopen(url, timeout=10) as response:
            return json.loads(response.read().decode())
    except Exception as e:
        print(f" [HTTP ERROR] {e}")
        return None
```

All API methods (`search_by_name`, `filter_by_category`, `filter_by_area`, `filter_by_ingredient`, `random_recipe`, `get_detail`, `get_categories`, `get_areas`, `get_ingredients`) route through the central `fetch()` method which handles errors without crashing the server.

RecipeFormatter Transforms raw API data into the shapes the client expects.

```
@staticmethod
def brief_list(meals: list, limit: int = 15) -> list:
    return [
        {'idMeal': m.get('idMeal', ''),
         'strMeal': m.get('strMeal', ''),
         'strMealThumb': m.get('strMealThumb', '')}
        for m in meals[:limit]
    ]
```

`brief_list` returns up to 15 results. `full_detail` combines TheMealDB's 20 separate ingredient fields into one clean list.

ReferenceCache Loads categories, areas, and ingredients once at startup and keeps them in memory.

```
def load(self) -> bool:
    self.categories = self._api.get_categories()
    self.areas = self._api.get_areas()
    self.ingredients = self._api.get_ingredients()
```

Since reference data rarely changes, it is cached at startup and served from memory to all clients without repeated API calls. Results are also saved to `reference_A4.json`.

ClientHandler Manages the full lifecycle of one connected client in its own thread.

```
def _dispatch(self, req_type: str, params: str) -> bool:
    handlers = {
        'GET_CATEGORIES': self._handle_get_categories,
        'SEARCH_BY_NAME': self._handle_search_by_name,
        'FILTER_BY_CATEGORY': self._handle_filter_by_category,
        'FILTER_BY_AREA': self._handle_filter_by_area,
        'FILTER_BY_INGREDIENT': self._handle_filter_by_ingredient,
        'RANDOM_RECIPE': self._handle_random_recipe,
        'GET_DETAIL': self._handle_get_detail,
    }
```

Uses a dictionary-based dispatcher to route request types to handler methods. Every recipe response is saved via `_save_and_send` before being sent to the client.

RecipeServer Owns the listening socket and spawns a daemon thread for each client.

```
while True:
    conn, addr = server_socket.accept()
    handler = ClientHandler(conn, self._cache)
    thread = threading.Thread(target=handler.run)
    thread.daemon = True
    thread.start()
```

`client.py`

The client script provides an interactive terminal menu for users to send requests to the server and display results.

Utilized packages: `socket`, `json`

Classes

Class	Responsibility
<code>MessageHelper</code>	Same length-prefixed protocol as server
<code>Display</code>	All terminal output — headers, recipe lists, full details
<code>InputHelper</code>	All validated user input — menus, prompts, pick lists
<code>BaseMenu</code>	Abstract base class — shared socket, display, and input helpers
<code>RecipesMenu</code>	Inherits <code>BaseMenu</code> — handles recipe search and filter screens
<code>ReferenceMenu</code>	Inherits <code>BaseMenu</code> — handles reference list screens
<code>MainMenu</code>	Inherits <code>BaseMenu</code> — top-level navigation between menus
<code>RecipeClient</code>	Manages connection lifecycle and launches the application

Additional Concept

Object-Oriented Programming (OOP)

The entire project is structured using OOP with a proper class hierarchy and clearly separated responsibilities across both scripts.

The Four OOP Concepts Applied

1. Encapsulation Internal attributes and methods are hidden using the `_` prefix. Only the public interface is exposed to the outside:

```
class ClientHandler:
    def __init__(self, conn, cache):
        self._conn      = conn      # private
        self._cache      = cache     # private
        self._username   = "unknown" # private

    def run(self) -> None:           # public — only entry point
        self._handshake()
        self._request_loop()
```

2. Inheritance The client menu system uses a hierarchy where all menus inherit shared functionality from `BaseMenu`:

```

class BaseMenu:
    def __init__(self, sock):
        self._sock = sock
        self._display = Display()
        self._input = InputHelper()

class RecipesMenu(BaseMenu):    # inherits all helpers from BaseMenu
    def run(self): ...

class ReferenceMenu(BaseMenu): # inherits all helpers from BaseMenu
    def run(self): ...

class MainMenu(BaseMenu):      # inherits all helpers from BaseMenu
    def run(self): ...

```

3. Polymorphism `MainMenu` calls `.run()` on any menu object through the same interface:

```

if choice == 1:
    RecipesMenu(self._sock).run()    # polymorphic call
elif choice == 2:
    ReferenceMenu(self._sock).run()  # polymorphic call

```

4. Abstraction `BaseMenu` defines a contract that all subclasses must implement:

```

class BaseMenu:
    def run(self) -> None:
        raise NotImplementedError("Subclasses must implement run()")

```

Acknowledgments

We would like to thank:

- **Dr. Mohammed Almeer** — for his guidance and support throughout this project
- **TheMealDB** (<https://www.themealdb.com>) — for providing the free recipe API used in this project
- **University of Bahrain** — for providing the resources and environment to complete this work

Conclusion

This project successfully implements a fully functional multi-threaded client-server recipe finder application. We implemented TCP socket programming, a length-prefixed messaging protocol, a hybrid data-handling pattern combining a startup reference cache with on-demand API fetching, and a complete Object-Oriented design with six server-side classes and eight client-side classes.

Through this project we gained hands-on experience with network programming, concurrent client handling, API integration, and structuring a real application using OOP principles. The challenges we faced — such as TCP fragmentation, thread safety, and JSON file path handling — deepened our understanding of how real networked systems work in practice.

Resources

- Python Socket Documentation: <https://docs.python.org/3/library/socket.html>
- Python Threading Documentation: <https://docs.python.org/3/library/threading.html>
- Python urllib Documentation: <https://docs.python.org/3/library/urllib.request.html>
- TheMealDB API Documentation: <https://www.themealdb.com/api.php>
- Python OOP Guide: <https://docs.python.org/3/tutorial/classes.html>